DBMS Project Implementation:

1. USER & ACCESS DOMAIN

`Customer`

- `customer_id` (PK): The surrogate key. Decouples the user's identity from changeable data like emails or phone numbers.
- `first_name`, `last_name`, `email`, `phone`: Core contact data. `email` is enforced as `UNIQUE` to act as a login anchor and prevent accidental duplicate profiles.
- `license_number` (UNIQUE): **Benefit:** The ultimate legal anchor. Government IDs are required for liability. Making it unique ensures that a banned driver cannot simply use a new email address to rent a car again.
- `risk_score`: **Benefit:** Allows the business to flag historically risky drivers without running massive historical queries across past damages.
- `is_blacklisted`: **Benefit:** A hard boolean stop at the database level that instantly prevents known bad actors from initiating a reservation.

`Branch`

- `branch_id` (PK): Uniquely identifies physical rental locations (like the HSR Layout branch).
- `branch_name`, `address`, `city`, `contact_phone`: Standard geographical and contact routing data.
- `total_rating_score` & `review_count`: **Benefit:** Controlled denormalization. By storing these here, the public catalog search loads instantly without needing to run heavy math across thousands of historical reviews.

`Customer_Violation`

- `violation_id` (PK): Identifies the specific infraction.
- `customer_id` (FK): Links the infraction to the user.
- `type`, `description`, `date_recorded`: **Benefit:** Isolates negative events (like late returns) into a dedicated log. This prevents the database from having to scan every single historical rental agreement just to find out if a customer is a repeat offender.

2. CATALOG & PRICING DOMAIN

`Vehicle_Category`

- `category_id` (PK): The abstract identifier for a class of cars (e.g., "Compact SUV").
- `category_name`, `passenger_capacity`, `baggage_capacity`: **Benefit:** 3NF Abstraction. By putting these here, we avoid typing "5 passengers" thousands of times

across the physical vehicle table, saving massive amounts of storage and preventing data inconsistencies.

`Rental_Pricing`

- `pricing_id` (PK): Uniquely identifies a specific price point.
- `category_id` (FK), `branch_id` (FK): Links the price to the category, allowing different branches to have different base rates.
- `daily_rate`, `weekly_rate`: The actual financial cost.
- `effective_from`, `effective_to`: **Benefit:** Slowly Changing Dimensions (SCD Type 2). This allows you to schedule holiday surges in advance while perfectly freezing historical prices, ensuring a receipt generated in January never recalculates based on July's prices.

3. FLEET & INVENTORY DOMAIN

`Vehicle`

- `vehicle_id` (PK): The ultimate identifier for the physical, depreciating asset.
- `category_id` (FK), `current_branch_id` (FK): Links the physical car to its abstract class and current physical location.
- `license_plate` (UNIQUE), `vin_number` (UNIQUE): **Benefit:** Absolute legal tracking. Ensures no two cars are ever confused in the system.
- `make`, `model`, `year`: **Benefit:** Granular depreciation tracking. Even if two cars are "Compact SUVs", knowing one is a 2024 model and one is a 2020 model allows for accurate maintenance predictions and asset sales.
- `current_mileage`, `current_status`: **Benefit:** Drives the operational state machine. `current_status` (Available, Rented, Maintenance) prevents double-booking at the physical level without deleting the car's existence.
- `total_rating_score` & `review_count`: **Benefit:** Denormalized read-optimization for lightning-fast vehicle quality sorting in the app.

`Fleet_Transfer`

- `transfer_id` (PK), `vehicle_id` (FK), `from_branch_id` (FK), `to_branch_id` (FK): **Benefit:** Maps exactly what car is moving where.
- `dispatched_at`, `arrived_at`, `status`: **Benefit:** Stops cars from "vanishing" from inventory while on a truck between cities, ensuring utilization reports account for transit time.

4. BOOKING & EXECUTION DOMAIN

`Reservation`

- `reservation_id` (PK), `customer_id` (FK), `category_id` (FK), `pickup_branch_id` (FK), `return_branch_id` (FK): **Benefit:** Captures the *intent*. Notice it locks a `category_id`, not a specific car. This is "Soft Allocation," ensuring if a specific car breaks, the reservation survives.
- `expected_pickup_datetime`, `expected_return_datetime`, `status`: Sets the chronological boundaries and the lifecycle state of the user's intent.

`Rental_Agreement`

- `agreement_id` (PK), `reservation_id` (FK, UNIQUE): **Benefit:** Captures the legal *execution*. The UNIQUE constraint ensures one intent translates to exactly one finalized contract.
- `agreement_date`, `agreed_daily_rate`, `terms_accepted`: Freezes the agreed-upon financials at the exact moment of signing, protecting the business legally.

`Rental_Vehicle_Assignment`

- `assignment_id` (PK), `agreement_id` (FK), `vehicle_id` (FK): **Benefit:** The physical bridging table. By separating this from the agreement, one rental contract can involve multiple physical cars if a breakdown swap occurs mid-trip.
- `start_datetime`, `end_datetime`: The exact timestamps of possession.
- `start_odometer`, `end_odometer`, `start_fuel_level`, `end_fuel_level`: **Benefit:** Operational baselines. Without these, the business cannot legally enforce mileage limits or charge for unrefilled gas tanks.

5. BILLING & LEDGER DOMAIN

`Payment_Transaction`

- `transaction_id` (PK), `reservation_id` (FK): Ties the money to the booking.
- `type`, `amount`, `transaction_date`: **Benefit:** The Append-Only Ledger. Instead of a single `balance_due` column that gets overwritten, every charge, refund, and penalty is a new row. This guarantees mathematically flawless revenue reporting and a perfect audit trail.

6. INSPECTION, MAINTENANCE & INSURANCE DOMAIN

`Vehicle_Inspection`

- `inspection_id` (PK), `assignment_id` (FK - Nullable), `vehicle_id` (FK), `type`, `inspector_id`, `inspection_timestamp`: **Benefit:** By making the

assignment nullable, this table handles both customer-driven inspections (pickup/return) and routine fleet checks in one unified pipeline.

`Damage_Report`

- `report_id` (PK), `inspection_id` (FK), `damage_type`, `description`, `severity`: **Benefit:** Granular liability. It ensures pre-existing scratches caught on a pickup inspection are not wrongly charged to the customer on the return inspection.

`Maintenance_Log`

- `log_id` (PK), `vehicle_id` (FK), `report_id` (FK - Nullable): **Benefit:** Links damages directly to the repair shop.
- `type`, `description`, `cost`, `status`, `date_logged`, `date_completed`: Tracks the total operational cost of keeping a specific asset on the road, critical for calculating ROI on the fleet.

`Insurance_Policy` & `Insurance_Claim`

- `policy_id` (PK), `vehicle_id` (FK), `provider_name`, `coverage_type`, `deductible`: Maps the overarching business protection to the physical asset.
- `claim_id` (PK), `report_id` (FK), `policy_id` (FK), `claim_amount`, `approved_amount`, `status`: **Benefit:** Ensures that when a `Damage_Report` costs the business money, there is a strict tracking mechanism for recovering those funds from the insurance provider.

7. FEEDBACK & REVIEWS DOMAIN

`Customer_Review`

- `review_id` (PK), `agreement_id` (FK, UNIQUE): **Benefit:** The "Verified Review" pattern. Tying this directly to the executed agreement guarantees zero fake reviews can be submitted.
- `vehicle_rating`, `branch_rating`, `comments`, `review_date`, `status`: **Benefit:** Separates the rating of the physical car from the rating of the customer service desk, allowing management to pinpoint exact operational failures.

Frequent Queries:

1. Available vehicles by category and branch (The "Whiteboard"

Version)

The Trick: Instead of a massive subquery, use `NOT IN`. It reads exactly like plain English: "Give me cars that are NOT IN the list of currently booked cars."

SQL

```
SELECT v.license_plate, vc.category_name
FROM Vehicle v
JOIN Vehicle_Category vc ON v.category_id = vc.category_id
WHERE v.current_branch_id = 1
  AND v.current_status = 'Available'
  AND v.vehicle_id NOT IN (
      SELECT vehicle_id FROM Rental_Vehicle_Assignment
      WHERE end_datetime IS NULL -- Car is currently out
  );
```

How to explain it: *"I select all available cars at the branch, and simply exclude any car ID that currently has an open assignment."*

2. VEHICLES OVERDUE FOR RETURN

The Trick: Ignore the maintenance part for the whiteboard. Just focus on the core logic: the expected date has passed, but the car hasn't been returned.

SQL

```
SELECT v.license_plate, res.expected_return_datetime
FROM Rental_Vehicle_Assignment rva
JOIN Rental_Agreement ra ON rva.agreement_id = ra.agreement_id
JOIN Reservation res ON ra.reservation_id = res.reservation_id
JOIN Vehicle v ON rva.vehicle_id = v.vehicle_id
WHERE rva.end_datetime IS NULL
  AND res.expected_return_datetime < CURRENT_TIMESTAMP;
```

How to explain it: *"I join the physical assignment to the reservation, and look for rows where the return date is in the past, but the actual return timestamp is still empty."*

3. REVENUE GENERATED BY CATEGORY AND BRANCH

The Trick: Because we built the `Branch_Daily_Metrics` view, this is already the simplest query in the system.

SQL

```
SELECT branch_id, category_id, SUM(total_revenue)
FROM Branch_Daily_Metrics
WHERE report_date >= '2026-06-01'
GROUP BY branch_id, category_id;
```

How to explain it: *"Because I decoupled analytical reads from the live ledger using a Materialized View, calculating monthly revenue is just a basic* `SUM` *and* `GROUP BY`*."*

4. CUSTOMERS WITH REPEATED LATE RETURNS OR DAMAGE

The Trick: Keep the `SELECT` clause small and focus entirely on the `HAVING` clause, which is what the professor is actually testing you on.

SQL

```
SELECT customer_id, COUNT(violation_id) AS total_violations
FROM Customer_Violation
WHERE type IN ('Late Return', 'Severe Damage')
GROUP BY customer_id
HAVING COUNT(violation_id) > 1;
```

How to explain it: *"I group the violation log by the customer ID, and use the* `HAVING` *clause to filter out anyone who doesn't have more than one strike on their record."*

5. AVERAGE RENTAL DURATION BY VEHICLE TYPE

The Trick: Skip the complex timestamp epoch math. Just write standard SQL date subtraction. The professor just wants to see that you know how to aggregate it.

SQL

```
SELECT vc.category_name, AVG(rva.end_datetime - rva.start_datetime) AS
avg_duration
FROM Rental_Vehicle_Assignment rva
JOIN Vehicle v ON rva.vehicle_id = v.vehicle_id
JOIN Vehicle_Category vc ON v.category_id = vc.category_id
WHERE rva.end_datetime IS NOT NULL
GROUP BY vc.category_name;
```

How to explain it: *"I calculate the difference between the actual start and end timestamps in the assignment table, then group those averages by the category name."*

6. FLEET UTILIZATION RATES

The Trick: Skip calculating the total fleet capacity dynamically. Just query the days rented from the metrics view.

SQL

```
SELECT branch_id, SUM(utilized_fleet_count) AS total_rented_days
FROM Branch_Daily_Metrics
WHERE report_date >= '2026-06-01'
GROUP BY branch_id;
```